
Module 5

Structure, Union, and Enumerated Data Type

5.1 Introduction

- ✓ **Structure** is basically a **user-defined data type** that can store related information (even of different data types) together.
- ✓ The major difference between a structure and an array is that an array can store only information of same data type.
- ✓ **A structure is a collection of variables under a single name. The variables within a structure are of different data types and each has a name that is used to select it from the structure.**
- ✓ “A Structure is a user defined data type, which is used to store the values of different data types together under the same name”.

5.1.1 Structure Declaration

- ✓ A structure is declared using the keyword **struct** followed by the structure name.
- ✓ All the variables of the structure are declared within the structure.
- ✓ A structure type is generally declared by using the following syntax:

```
struct struct-name
{
    data_type var-name;
    data_type var-name;
    .....
};
```

For example:

```
struct student
{
    int r_no;
    char name[20];
    char course[20];
    float fees;
};
```

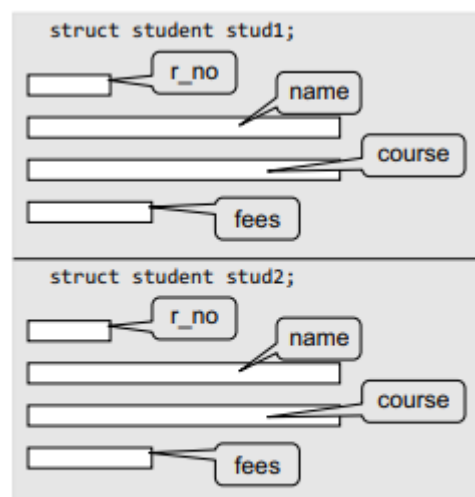


Figure 5.1 Memory allocation for a structure variable

- ✓ A variable of structure student can be defined by writing:
struct student stud1;
struct student is a data type and **stud1** is a variable.

- ✓ In the following syntax, the variables are declared at the time of structure declaration.

```
struct student
{
    int r_no;
    char name[20];
    char course[20];
    float fees;
```

```
} stud1, stud2;
```

`stud1` and `stud2` of the structure student.

5.1.2 Typedef Declarations

- ✓ The typedef (derived from type definition) **keyword enables the programmer to create a new data type name by using an existing data type alternate name is given to a known data type.**
- ✓ The general syntax of using the typedef keyword is given as:

typedef existing_data_type new_data_type;

For example: typedef int INTEGER;

then INTEGER is the new name of data type int.

- ✓ To declare variables using the new data type name, precede the variable name with the data type name (new).
- ✓ Therefore, to define an integer variable, INTEGER num=5;

For example, consider the following declaration:

typedef struct student

```
{
    int r_no;
    char name[20];
    char course[20];
    float fees;
```

```
};
```

5.1.3 Initialization of Structures

- ✓ Initializing a structure means **assigning some constants to the members of the structure.**
- ✓ When the **user does not explicitly initialize the structure, then C automatically does it.**
- ✓ For **int and float members, the values are initialized to zero, and char and string members are initialized to '\0' by default.**
- ✓ The initializers are enclosed in braces and are separated by commas.
- ✓ The general syntax to initialize a structure variable is as follows:

struct struct_name

```
{
    data_type member_name1;
    data_type member_name2;
    data_type member_name3;
```

.....

}struct_var = {constant1, constant2, constant3,...};

or

struct struct_name

```
{
    data_type member_name1;
    data_type member_name2;
    data_type member_name3;
```

.....

```
};
```

struct struct_name struct_var = {constant1, constant2, constant 3,...};

- ✓ For example, we can initialize a student structure by writing,

struct student

```
{
    int r_no;
    char name[20];
    char course[20];
    float fees;
}stud1 = {01, "Rahul", "BCA", 45000};
```

Or,

by writing,

```
struct student stud1 = {01, "Rahul", "BCA", 45000};
```

struct student stud1 = {01, "Rahul", "BCA", 45000};				struct student stud2 = {07, "Rajiv"};			
01	Rahul	BCA	45000	07	Rajiv	\0	0.0
r_no	name	course	fees	r_no	name	course	fees

5.1.4 Accessing the Members of a Structure

- ✓ A structure member variable is generally accessed using a **'.'** (dot) operator. The syntax of accessing a structure or a member of a structure can be given as:

struct_var. member_name

For example, to assign values to the individual data members of the structure variable stud1, we may write

```
stud1.r_no = 01;
```

```
stud1.name = "Rahul";
```

```
stud1.course = "BCA";
```

```
stud1.fees = 45000;
```

- ✓ To input values for data members of the structure variable stud1, we may write

```
scanf("%d", &stud1.r_no);
```

```
scanf("%s", stud1.name);
```

- ✓ Similarly, to print the values of structure variable stud1, we may write

```
printf("%s", stud1.course);
```

```
printf("%f", stud1.fees);
```

5.1.5 Copying and Comparing Structures

- ✓ We can assign a structure to another structure of the same type:

- ✓ Then to assign one structure variable to another, we will write

```
stud2 = stud1;
```

- ✓ C does not permit comparison of one structure variable with another. However, individual members of one structure can be compared with individual members of another structure.

- ✓ For example, to compare the fees of two students, we will write

```
if(stud1.fees > stud2.fees) //to check if fees of stud1 is greater than stud2
```

struct student stud1 = {01, "Rahul", "BCA", 45000};			
01	Rahul	BCA	45000
r_no	name	course	fees

struct student stud2 = stud1;			
01	Rahul	BCA	45000
r_no	name	course	fees

5.1.6 Finding the Size of the structures

- ✓ There are two different ways through which we can find the number of bytes a structure will occupy in the memory.

1. Simple Addition

- ✓ In this techniques, make a list of all data types and add the memory required by each.
- ✓ Consider a simple structure of an employee

```
struct Employee
```

```
{
    int emp_id;
    char name[20];
    double salary;
    char designation[20];
    float experience;
};
```

Size = size of emp_id+ size of name + size of salary + size of designation + size of experience

Size of emp_id = 2

Size of name = 20* size of character

Size of salary = 8

Size of designation = 20 * Size of character

Size of experience = 4

Therefore, Size = 2 + 20*1 + 8 + 20 * 1 + 4

= 2 + 20 + 8 + 20 + 4

= 54bytes

2. Using sizeof operator

✓ The **sizeof** operator is used to calculate the **size of a data type, variable, or an expression.**

✓ This operator can be used as follows:

sizeof(struct_name);

Ex:

```
#include<stdio.h>
```

```
struct employee
```

```
{
    int emp_id;
    char name[10];
    double salary;
    char designation [20];
    float experience;
};
```

```
void main()
```

```
{
    struct employee e;
    printf("%d", sizeof(e));
}
```

Example: C program to read and display the student details using structures.

```
#include<stdio.h>
```

```
struct student
```

```
{
    int rnum;
    char name[20];
    int marks;
}s[60];
```

```
void main()
```

```
{
    int i,n;
    printf("Enter the number of students");
```

```

scanf("%d",&n);

printf("\nEnter the roll number, Name , Marks\n");
for(i=1;i<=n;i++)
{
    printf("\nStudent %d details\n",i);
    scanf("%d",&s[i].rnum);
    scanf("%s",s[i].name);
    scanf("%d",&s[i].marks);
}
printf("\nStudent Details are:");
printf("\nRoll_number\tName\tMarks");
for(i=1;i<=n;i++)
    printf("\n%d\t\t\t%s\t%d\n",s[i].rnum,s[i].name,s[i].marks);

}

```

Output:

Enter the number of students: 5

Enter the roll number,

Name, Marks and Grade of

Student 1 details

14

Dhavan

89

Student 2 details

15

Karan

55

Student 3 details

11

Deepa

45

Student 4 details

12

Lakshmi

35

Student 5 details

10

Soma

68

Student Details are:

Roll_number	Name	Marks
14	Dhavan	89
15	Karan	55
11	Deepa	45
12	Lakshmi	35
10	Soma	68

5.2 Structures and Functions

- ✓ A function may access the members of a structure in three ways.

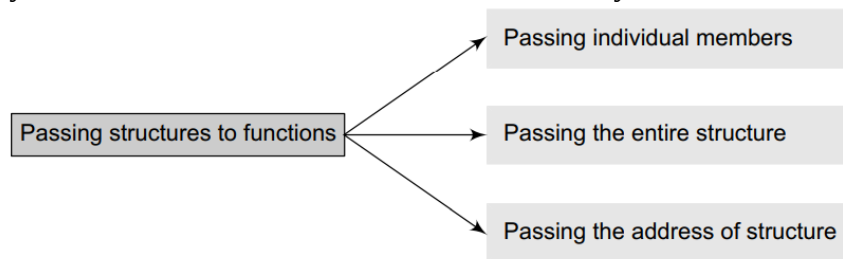


Figure 5.4 Different ways of passing structures to functions

1. Passing Individual Members

- ✓ To pass any individual member of a structure to a function, we must use the direct selection operator to refer to the individual members.

```
#include<stdio.h>
typedef struct
{
    int x;
    int y;
}POINT;

void display(int, int);

void main()
{
    POINT p1 = {2, 3};
    display(p1.x, p1.y);
}

void display(int a, int b)
{
    printf(" The coordinates of the point are: %d %d", a, b);
}
```

Output: The coordinates of the point are: 2 3

2. Passing the Entire Structure

- ✓ The entire structure can be passed as a function argument.
- ✓ When a structure is passed as an argument, it is passed using the call by value method, i.e., a copy of each member of the structure is made.
- ✓ The general syntax for passing a structure to a function and returning a structure can be given as,
struct struct_name func_name(struct struct_name struct_var);

```
#include <stdio.h>
typedef struct
{
    int x;
    int y;
}POINT;

void display(POINT);
```

```

void main()
{
    POINT p1 = {2, 3};
    display(p1);
}

void display(POINT p)
{
    printf("The coordinates of the point are: %d %d", p.x, p.y);
}

```

3. Passing Structures through Pointers

- ✓ The syntax to declare a pointer to a structure can be given as,

```

struct struct_name
{
    data_type member_name1;
    data_type member_name2;
    data_type member_name3;
    .....
}*ptr;

or,
    struct struct_name *ptr;

```

- ✓ For the student structure, we can declare a pointer variable by writing

struct student *ptr_stud, stud;

And to assign the address, we will write

ptr_stud = &stud;

- ✓ To access the members of a structure, we can write

/ get the structure, then select a member */*

(*ptr_stud).roll_no;

This operator is known as ‘pointing-to’ operator (->). It can be used as:

ptr_stud ->roll_no = 01;

Example: Write a program using pointer to structure to initialize the members in the structure.

```
#include<stdio.h>
```

```
struct student
```

```
{
    int r_no;
    char name[20];
    char course[20];
    float fees;
};
```

```
void main()
```

```
{
    struct student stud1, *ptr_stud1;
    ptr_stud1 = &stud1;
    ptr_stud1->r_no = 01;
    strcpy(ptr_stud1->name, "Rahul");
    strcpy(ptr_stud1->course, "BCA");
}
```

```

ptr_stud1->fees = 45000;
printf("\n DETAILS OF STUDENT");
printf("\n -----");
printf("\n ROLL NUMBER = %d", ptr_stud1->r_no);
printf("\n NAME = %s", ptr_stud1->name);
printf("\n COURSE = %s", ptr_stud1->course);
printf("\n FEES = %f", ptr_stud1->fees);
}

```

5.3 Union

- ✓ Like structure, a **union is a collection of variables of different data types**. The only difference between a structure and a union is that in case of unions, **you can only store information in one field at any one time**.
- ✓ To better understand union, think of it as a chunk of memory that is used to store variables of different types. When a new value is assigned to a field, the existing data is replaced with the new data.

5.3.1 Declaring a Union

- ✓ The syntax for declaring a union is same as that of declaring a structure.

```

union union-name
{
    data_type var-name;
    data_type var-name;
    .....
};

```

- ✓ Again, the typedef keyword can be used to simplify the declaration of union variables.
- ✓ The most important thing to remember about a union is that the size of an **union is the size of its largest field**. This is because a sufficient number of bytes must be reserved to store the largest sized field.

5.3.2 Accessing a Member of a Union

- ✓ A member of a union can be accessed using the same syntax as that of a structure.
- ✓ To access the fields of a union, use the **dot operator(.)**.
- ✓ That is the union variable name followed by the dot operator followed by the member name.

5.3.3 Initializing Unions

- ✓ It is an error to initialize any other union member except the first member.
- ✓ A striking difference between a structure and a union is that in case of a union, the fields share the same memory space, so fresh data replaces any existing data. Look at the code given below and observe the difference between a structure and union when their fields are to be initialized.

```

#include<stdio.h>
typedef struct POINT1
{
    int x, y;
};
typedef union POINT2
{
    int x;
    int y;
};

```

```

void main()
{
    POINT1 P1 = {2,3};
    // POINT2 P2 = {4,5}; Illegal with union
    POINT2 P2;
    P2.x = 4;
    P2.y = 5;
    printf("\n The co-ordinates of P1 are %d and %d", P1.x, P1.y);
    printf("\n The co-ordinates of P2 are %d and %d", P2.x, P2.y);
}

```

Output:

The co-ordinates of P1 are 2 and 3

The co-ordinates of P2 are 5 and 5

5.4 Unions inside Structures

- ✓ Union can be very useful when declared inside a structure. Consider an example in which you want a field of a structure to contain a string or an integer, depending on what the user specifies. The following code illustrates such a scenario.

```

struct student
{
    union
    {
        char name[20];
        int roll_no;
    };
    int marks;
};

void main()
{
    struct student stud;
    char choice;
    printf("\n You can enter the name or roll number of the student");
    printf("\n Do you want to enter the name? (Yes or No) : ");
    scanf(&choice);
    if(choice=='y' || choice=='Y')
    {
        printf("\n Enter the name : ");
        scanf("%s",&stud.name);
    }
    else
    {
        printf("\n Enter the roll number : ");
        scanf("%d", &stud.roll_no);
    }
    printf("\n Enter the marks : ");
    scanf("%d", &stud.marks);
    if(choice=='y' || choice=='Y')
        printf("\n Name : %s ", stud.name);
    else
        printf("\n Roll Number : %d ", stud.roll_no);
        printf("\n Marks : %d", stud.marks);
}

```

}

5.5 Enumerated Data Types

- ✓ The enumerated data type is a **user defined type** based on the **standard integer type**.
- ✓ An enumeration consists of a set of named integer constants. That is, in an enumerated type, **each integer value is assigned an identifier**. This identifier (also known as an enumeration constant) can be used as **symbolic names to make the program more readable**.
- ✓ To define enumerated data types, **enum** keyword is used.
- ✓ Enumerations create new data types to contain values that are not limited to the values fundamental data types may take. The syntax of creating an enumerated data type can be given as below.

enum enumeration_name{ identifier1, identifier2,, identifier n };

- ✓ Consider the example given below which creates a new type of variable called COLORS to store colors constants.

enum COLORS {RED, BLUE, BLACK, GREEN, YELLOW, PURPLE, WHITE};

- ✓ In case you do not assign any value to a constant, the default value for the first one in the list - RED (in our case), has the value of 0. The rest of the undefined constants have a value 1 more than its previous one. So in our example,

RED = 0, BLUE = 1, BLACK = 2, GREEN = 3, YELLOW = 4, PURPLE = 5, WHITE = 6

- ✓ If you want to explicitly assign values to these integer constants, then you should specifically mention those values as shown below.

enum COLORS {RED = 2, BLUE, BLACK = 5, GREEN = 7, YELLOW, PURPLE, WHITE = 15};

- ✓ As a result of the above statement, now

RED = 2, BLUE = 3, BLACK = 5, GREEN = 7, YELLOW = 8, PURPLE = 9, WHITE = 15

5.5.1 Enum Variables

- ✓ The syntax for declaring a variable of an enumerated data type can be given as,

enum enumeration_name variable_name;

- ✓ So to create a variable of COLORS, we may write:

enum COLORS bg_color;

- ✓ Another way to declare a variable can be as illustrated in the statement below.

enum COLORS {RED, BLUE, BLACK, GREEN, YELLOW, PURPLE, WHITE}bg_color, fore_color;

5.5.2 Using the Typedef Keyword

- ✓ C permits to use typedef keyword for enumerated data types. For ex, if we write

typedef enum COLORS color;

- ✓ Then, we can straight-away declare variables by writing

color forecolor = RED;

5.5.3 Assigning values to Enumerated Variables

- ✓ Once the enumerated variable has been declared, **values can be stored in it**.
- ✓ However, an enumerated variable can hold only declared values for the type.
- ✓ For example, to assign the color black to the back ground color, we will write,

bg_color = BLACK;

- ✓ Once an enumerated variable has been assigned a value, we can store its value in another variable of the same type as shown below.

enum COLORS bg_color, border_color;

bg_color = BLACK;

border_color = bg_color;

5.5.4 Enumeration Type Conversion

- ✓ Enumerated types can be **implicitly or explicitly cast**.
- ✓ For ex, the compiler can implicitly cast an enumerated type to an integer when required.
- ✓ However, when we implicitly cast an integer to an enumerated type, the compiler will either generate an error or warning message.
- ✓ To understand this, answer one question. If we write:

```
enum COLORS{RED, BLUE, BLACK, GREEN, YELLOW, PURPLE, WHITE};  
  
enum COLORS c;  
  
c = BLACK + WHITE;
```

- ✓ Here, c is an enumerate data type variable. If we write, c = BLACK + WHITE, then logically, it should be 2 + 6 = 8; which is basically a value of type int. However, the left hand side of the assignment operator is of the type enumCOLORS. SO the statement would complain an error.
- ✓ To remove the error, you can do either of two things. First, declare c to be an int.
- ✓ Second, cast the right hand side in the following manner. :

```
c = enum COLORS(BLACK + WHITE);
```

5.5.5 Comparing Enumerated Types

- ✓ C also allows using comparison operators on enumerated data type. Look at the following statements which illustrate this concept.

```
bg_color = (enum COLORS)6;  
if(bg_color == WHITE)  
    fore_color = BLUE;  
fore_color = BLACK;  
if(bg_color == fore_color)  
    printf("\n NOT VISIBLE");
```

- ✓ Since enumerated types are derived from integer type, they can be used in a switch-case statement.

```
enum {RED, BLUE, BLACK, GREEN, YELLOW, PURPLE, WHITE}bg_color;  
switch(bg_color)  
{  
    case RED:  
    case BLUE:  
    case GREEN:  
        printf("\n It is a primary color");  
        break;  
    case default:  
        printf("\n It is not a primary color");  
        break;  
}
```

5.5.6 Input/Output Operations on Enumerated Types

- ✓ Since enumerated types are derived types, they cannot be read or written using formatted I/O functions available in C. When we read or write an enumerated type, we read/write it as an integer. The compiler would implicitly do the type conversion as discussed earlier.

```
enum COLORS(RED, BLUE, BLACK, GREEN, YELLOW, PURPLE, WHITE);  
enum COLORS c;  
c = RED;  
printf("\n Color = %d", c);
```

5.5.7 Differences between structure and union

Structure	Union
The struct keyword is used to define it.	The union keyword is used to define it.
Syntax: struct struct-name { data_type var-name; data_type var-name; };	Syntax: union union-name { data_type var-name; data_type var-name; };
Example: struct student { int r_no; char name[20]; char course[20]; float fees; };	Example: union student { int r_no; char name[20]; char course[20]; float fees; };
A variable of structure student can be defined by writing: struct student stud1;	A variable of union student can be defined by writing: union student stud1;
Several members of a structure can be initialized at once. Ex: struct student stud1 = {01, "Rahul", "BCA", 45000};	Individual members of a structure can be initialized one by one. Ex: union student stud1; stud1. r_no=01; stud1. name= "Rahul"; stud1. course="BCA"; stud1. fees =45000;
Each member within structure is assigned unique storage area of location.	The members or fields share the same memory space, so fresh data replaces any existing data.
The size of the structure is the sum of all members or fields.	The size of the union is the size of the largest member or field.
Altering the value of a member will not affect other members of the structure.	Altering any value of a member will alter other member values.